

Struct Memory Lab

Person A — Import & Layout

Sunum öncesi derinlemesine rehber: C++ struct bellek yerleşimi, ayrıştırma, hizalama matematiği, optimizasyon ve canlı görselleştirme. Mantık, kod ve sunum Q&A bir arada.

Bu belge kendi slice'ını (parse + layout + düzenleyici/görsel) kod, mantık ve tasarım kararlarıyla açıklar; jurinin sorabileceği sorulara hazırlar.

1. Proje ve Mimari Genel Bakış

Struct Memory Lab, bir C++ struct'ını alıp belleği nasıl kapladığını görselleştiren, düzenlemeye ve zaman içinde versiyonlamaya izin veren bir araçtır. Uygulama bir **boru hattı** (pipeline) gibi düşünülebilir:

```
C++ metni -> parseCpp -> StructModel -> computeLayout -> bellek haritası
                |                               |
                (duzenle/surukle)               (canli, zoom'lanabilir gorsel)
```

Proje iki kişilik bir staj çalışması ve **dikey slice** modeliyle bölünmüştür: kimse 'sadece frontend' ya da 'sadece backend' yapmaz; herkes bir özelliği **uçtan uca** (mantık + arayüz) sahiplenir.

İki slice

	Person A — Import & Layout	Person B — Versioning & Export
Backend	parser.ts, layout.ts	versioning, diff, compatibility, exporter
Frontend	ImportBox, FieldEditor, LayoutVisualizer	VersionPanel, DiffView, WarningsPanel, ExportBox

Bu belge **Person A** slice'ını anlatır. İki slice'ın buluşma noktası iki paylaşımlı dosyadır: **types.ts** (veri sözleşmesi) ve **useStructStore.ts** (ortak Zustand hafızası).

Teknolojiler ve neden

- **Next.js + React** — bileşen tabanlı UI; sayfa iki panele bölünmüş.
- **TypeScript** — sözleşmeyi (types.ts) derleme zamanında zorlar; iki slice uyumlu kalır.
- **Zustand** — hafif global state; iki slice'ın buluşma noktası.
- **dnd-kit** — erişilebilir sürükle-bırak (fare + klavye).
- **vitest** — saf mantığı (parse/layout/optimize) hızlı test eder.
- **Tailwind** — yardımcı sınıflarla hızlı stil.

Dosya haritası (Person A)

```
src/
types.ts          PAYLASIMLI - veri sozlesmesi + fonksiyon imzalari
store/useStructStore.ts PAYLASIMLI - ortak state (currentModel, versions)
engine/
  parser.ts       A   C++ metni -> StructModel
  layout.ts       A   offset/padding/size hesabi (+ alignUp)
  segments.ts     A   LayoutResult -> çizilebilir segmentler
  optimizer.ts    A   alanlari dizip padding'i azaltir
  validate.ts     A   alan adi dogrulamasi
components/
  ImportBox.tsx   A   yapistir + Parse + hata gosterimi
  FieldEditor.tsx A   isim/tip/ekle/sil + surukle-birak + optimize
  LayoutVisualizer.tsx A orantili, zoom'lanabilir bellek haritasi
```

2. Sözleşme — types.ts

Sözleşme, iki slice'ın birbirini **beklemeden** çalışabilmesini sağlayan anlaşmadır: veri şekilleri ve fonksiyon imzaları. Kural: bu dosya yalnızca birlikte değiştirilir.

Temel tipler

```
export interface Field {  
  id: string;           // STABIL kimlik (drag-drop + diff için şart)  
  name: string;  
  type: CppPrimitive; // 'uint32_t' | 'bool' | 'double' | ...  
  arrayLength: number; // tekil alan için 1, dizi için > 1  
}
```

```
export interface StructModel { name: string; fields: Field[]; }
```

Neden her alanın bir id'si var? Çünkü sürükle-bırak ve diff, alanı ismi değişse bile takip etmek zorunda. Index kırılğan (sıra değişince kayar), isim değişebilir; id sabittir. Bu, ileride dnd-kit'in SortableContext'inin de tam ihtiyacı olan şey.

Hesaplama çıktıları

```
export interface FieldLayout {  
  fieldId, name, type;  
  offset: number;           // struct basından byte  
  size: number;             // bu alanın kapladığı (dizi dahil)  
  paddingBefore: number; // bu alandan ONCE eklenen boş byte  
}  
export interface LayoutResult {  
  fields: FieldLayout[];  
  totalSize; // sizeof(struct) - tail padding dahil  
  alignment; // struct hizalaması (en büyük alan hizalaması)  
  totalPadding; // toplam boş giden byte  
}
```

TYPE_INFO — tek doğruluk kaynağı

Her primitive tipin byte cinsinden boyutu ve hizalaması burada tanımlı. Bu tablo **iki yerde** kullanılır: computeLayout (hesap) ve parseCpp ('bu tip geçerli mi?' kapısı).

Tip	Boyut (byte)	Hizalama
bool, char, int8_t, uint8_t	1	1
int16_t, uint16_t	2	2
int32_t, uint32_t, float	4	4
int64_t, uint64_t, double	8	8

Fonksiyon imzaları da burada (ParseCpp, ComputeLayout, ...). Gerçek gövdeler engine/ altındadır; böylece 'ne' (imza) ile 'nasıl' (uygulama) ayrılır.

3. Ortak Hafıza — Zustand Store

İki slice'ın buluşma noktası. Person A **currentModel**'i yazar/düzenler; Person B onu okuyup **versions**'a snapshot kaydeder. Store'un şekli birlikte değişir; herkes kendi action gövdesini doldurur.

```
export const useStructStore = create<StructState>((set, get) => ({
  currentModel: initialModel,
  versions: [],
  // PERSON A:
  setModel, setStructName, addField, updateField, removeField, reorderFields,
  // PERSON B:
  saveVersion, loadVersion,
}));
```

reorderFields — sürüklemenin veri tarafı

```
reorderFields: (fromIndex, toIndex) => set((s) => {
  const fields = [...s.currentModel.fields];
  const [moved] = fields.splice(fromIndex, 1); // çıkar
  fields.splice(toIndex, 0, moved);           // yeni yere koy
  return { currentModel: { ...s.currentModel, fields } };
}),
```

Önemli ders: sürükleme mantığı store'da, saf ve test edilebilir. dnd-kit sadece 'şunu şuraya taşı' der; nasıl taşınacağını store bilir. UI ile mantığın ayrılması.

Hydration ve sabit başlangıç id'leri

id üretici **Date.now()** kullanır; bu sunucuda (SSR) ve tarayıcıda farklı değer üretir → React **hydration uyumsuzluğu**. Çözüm: **başlangıç** alanlarına sabit id verdik ('f_id' gibi). Çalışma anında (client) eklenen alanlar makeld() kullanabilir, sorun değil.

4. parseCpp — Ayırıştırma (Parsing)

Parsing = serbest metni anlamlı parçalara bölmek. parseCpp, computeLayout'un tersi yönde ilk adımdır: yapııştırılan C++'ı yapılandırılmış StructModel'e çevirir. Üç katman:

- **Katman 1 — Bloğu bul.** 'struct <isim> { <gövde> }' kalıbını yakala.
- **Katman 2 — Gövdeyi alanlara böl.** ';' ile parçalara ayır.
- **Katman 3 — Her alanı ayırıştır.** '<tip> <isim>[N]?' deseninden tip/isim/dizi çıkar.

```
export const parseCpp = (code) => {  
  const clean = stripComments(code); // // ve /* */ temizle  
  const block = clean.match(/struct\s+([A-Za-z_]\w*)\s*\{([ \s\S]*?)\}/);  
  if (!block) throw new Error('Gecerli bir struct bulunamadi...');  
  const [, name, body] = block;  
  const fields = [];  
  for (const raw of body.split(';')) {  
    const decl = raw.trim(); if (!decl) continue;  
    const m = decl.match(FIELD_RE);  
    if (!m) throw new Error(`Alan cozumlenemedi: "${decl}"`);  
    const [, rawType, fieldName, len] = m;  
    const type = resolveType(rawType); // alias -&gt; kanonik  
    if (!type) throw new Error(`Bilinmeyen tip: "${rawType}"`);  
    fields.push({ id: makeId('f'), name: fieldName, type,  
      arrayLength: len ? parseInt(len,10) : 1 });  
  }  
  return { name, fields };  
};
```

Regex'i okumak

FIELD_RE = `/^([A-Za-z_][\w\s]*?)\s+([A-Za-z_]\w*)\s*(?:\[s*(\d+)\s*\])?$/`

- Grup 1 (tip): harf/_ ile başlar, boşluk içerebilir ('unsigned int'). **Non-greedy** (*) — mümkün en azını alır.
- Grup 2 (isim): son tanımlayıcı; non-greedy grup 1 sayesinde her zaman alan adını yakalar.
- Grup 3 (dizi): opsiyonel '[N]'; (?:...)? = yakalamayan opsiyonel grup.

Tip alias'ları ve hata kanalı

Gerçek C++'a yaklaşmak için **int**, **unsigned int**, **short**, **long**, **char** gibi yazımları kanonik sabit-genişlikli tiplere çeviririz (resolveType). parseCpp **hata fırlatır**; ImportBox bu hatayı yakalayıp kullanıcıya kırmızı mesaj gösterir — mantık katmanı UI'dan habersiz kalır.

5. computeLayout — Hizalama Matematiği

Projenin kalbi ve en çok soru gelecek kısım. Bellek tek boyutlu bir bant; alanları soldan sağa dizeriz ama **4 kurala** uyarak. Fonksiyon tamamen **deterministik** — bu yüzden test etmesi kolay.

Dört kural

- **1. Boyut:** eleman boyutu × dizi uzunluğu.
- **2. Hizalama:** alan, offset'i kendi align'nının KATI olan yere konur. (double yalnızca 0, 8, 16... offset'ine).
- **3. Padding:** hizalamak için araya boş byte eklenir.
- **4. Tail padding:** struct align = max(alan align'ları); toplam boyut bu değerin katına yukarı yuvarlanır.

Tek matematiksel hile **yukarı yuvarlama**: $\text{alignUp}(x, a) = \text{ceil}(x/a) * a$. Örneğin $\text{alignUp}(5, 8) = 8$, $\text{alignUp}(8, 8) = 8$, $\text{alignUp}(9, 8) = 16$. Bir offset'in hizalı olup olmadığı ise $\text{offset} \% \text{align} === 0$ ile bulunur.

```
export const computeLayout = (model) => {  
  const fields = []; let offset = 0; let maxAlign = 1;  
  for (const f of model.fields) {  
    const { size: elemSize, align } = TYPE_INFO[f.type];  
    const size = elemSize * Math.max(1, f.arrayLength);  
    const aligned = alignUp(offset, align); // kural 2  
    const paddingBefore = aligned - offset; // kural 3  
    fields.push({ fieldId: f.id, name: f.name, type: f.type,  
      offset: aligned, size, paddingBefore });  
    offset = aligned + size;  
    maxAlign = Math.max(maxAlign, align);  
  }  
  const totalSize = alignUp(offset, maxAlign); // kural 4 (tail padding)  
  const usedBytes = fields.reduce((s, f) => s + f.size, 0);  
  return { fields, totalSize, alignment: maxAlign,  
    totalPadding: totalSize - usedBytes };  
};
```

Çalışmış örnek: Player

```
struct Player { uint32_t id; bool alive; double health; };
```

Alan	Tip (hizalama)	Hesap	Offset	Kapladığı
id	uint32_t (4)	0, 0%4=0	0	0-3
alive	bool (1)	4, 4%1=0	4	4
health	double (8)	5 & rarr; 5%8≠0 & rarr; 8 (5,6,7 padding)	8	8-15

Bitiş offset'i 16; struct align 8; $16 \% 8 = 0 \rightarrow$ tail padding yok. **sizeof = 16, padding = 3** (sadece 5,6,7. byte'lar).

Tail padding NEDEN var?

Çünkü **diziler**. Bir Player arr[2] düşün: arr[0] offset 0'da başlar, arr[1] ise sizeof(Player) offset'inde. arr[1]'in içindeki **health** (double) yine 8'in katı bir offset'te olmalı. Bu garanti için sizeof, struct'ın hizalamasının (maxAlign) katı olmak zorunda. Yani senin özetin doğru: **maxAlign'ı tutarız ve toplam boyutu maxAlign'a tamamlarız** — 'son alan'a değil, **toplam boyuta** bakıp maxAlign'ın katı

değilse yukarı yuvarlarız.

Küçük düzeltme: tetikleyici 'son alan maxAlign'dan küçükse' değil; toplam boyut zaten maxAlign'ın katı değilse tail padding eklenir. Player'da toplam 16 ve $16\%8=0$ olduğu için tail padding YOK; ama örn. {uint32_t; bool} 5'te biter, 4'e değil 4'ün katı 8'e yuvarlanır → 3 tail padding.

6. toSegments + LayoutVisualizer

computeLayout sana **sayılar** verir; görselleştiricinin işi bunları **orantılı bir resme** çevirmek. Anahtar desen: **türet-sonra-çiz**. Saf bir fonksiyon (toSegments) çizilecek segment listesini üretir; React bileşeni o listeyi aptalca div'lere map'ler. Böylece zor kısım test edilebilir, bileşen basit kalır.

İki fikir

- **Padding'i görünür yap:** computeLayout padding'i alanların içinde bir sayı (paddingBefore) olarak tutar; toSegments bunu ayrı **gri bloka** çevirir, sonda da tail padding bloklarını ekler.
- **Genişlik = byte ile orantılı:** 8 byte'lık health, 1 byte'lık alive'dan 8 kat geniş çizilir.

```
export function toSegments(layout) {
  const segments = []; let colorIndex = 0;
  for (const f of layout.fields) {
    if (f.paddingBefore > 0)
      segments.push({ kind: 'padding', offset: f.offset - f.paddingBefore,
                      size: f.paddingBefore });
    segments.push({ kind: 'field', offset: f.offset, size: f.size,
                    name: f.name, type: f.type, colorIndex: colorIndex++ });
  }
  const last = layout.fields[layout.fields.length - 1];
  const usedEnd = last ? last.offset + last.size : 0;
  const tail = layout.totalSize - usedEnd;
  if (tail > 0) segments.push({ kind: 'padding', offset: usedEnd, size: tail });
  return segments;
}
```

Güzel bir **değişmez (invariant)**: tüm segment boyutlarının toplamı = totalSize. Bunu test ediyoruz.

Görselin özellikleri

- **Zoom (px/byte):** bir slider blok genişliklerini ölççekler; taşınca yatay kaydırma devreye girer.
- **Renk legend'i:** her alanın rengi + offset/boyutu.
- **Canlılık:** düzenle/sürükle/optimize et → harita anında yeniden çizilir.

7. optimizeStruct — Padding'i Azaltmak

Sezgi: padding, küçük hizalamalı bir alandan sonra büyük hizalamalı biri geldiğinde oluşur (büyük olan hizalı offset'e zıplar, araya boşluk girer). Çözüm: alanları **hizalamaya göre büyükten küçüğe** dizmek. Böylece her alan bir öncekinin bittiği yere çoğunlukla tam oturur.

```
export function optimizeStruct(model) {
  const fields = model.fields
    .map((f, i) => ({ f, i }))          // özgün index'i sakla
    .sort((a, b) => {
      const da = TYPE_INFO[a.f.type].align, db = TYPE_INFO[b.f.type].align;
      if (da !== db) return db - da;    // hizalama büyükten küçüğe
      return a.i - b.i;                // esitlikte özgün sıra (stabil)
    })
    .map((x) => x.f);
  return { ...model, fields };
}
```

Örnek: before / after

Sıralama	Alanlar	sizeof	padding
Özgün	bool, double, bool	24	14
Optimize	double, bool, bool	16	6

Önemli detay: optimizeStruct **saftır** (özgün modeli değiştirmez) ve **stabildir** (aynı hizalamadakilerin sırasını korur). Player gibi zaten iyi dizilmiş struct'larda sizeof değişmez — bu beklenen davranış.

8. validateModel — Doğrulama

Kullanıcıyı engellemeden uyarıyan saf bir fonksiyon: boş ad, geçersiz C++ tanımlayıcısı (harf/_ ile başlamalı) ve yinelenen ad. UI bu listeyi sarı uyarı olarak gösterir.

```
const IDENTIFIER = /^[A-Za-z_]\w*$/;
export function validateModel(model) {
  const issues = []; const counts = new Map();
  for (const f of model.fields) {
    const name = f.name.trim();
    if (!name) issues.push('Bos alan adi var.');
```

```
    else if (!IDENTIFIER.test(name)) issues.push(`Gecersiz alan adi: "${f.name}"`);
    if (name) counts.set(name, (counts.get(name) ?? 0) + 1);
  }
  for (const [name, c] of counts)
    if (c > 1) issues.push(`Yinelenen alan adi: "${name}" (${c} kez).`);
  return [...new Set(issues)]; // ayni mesaj tekrar etmesin
}
```

9. FieldEditor & Drag-Drop (dnd-kit)

dnd-kit, sürüklemenin görsel kısmını (fareyi takip, animasyon) halleder; sen sadece **bırakınca veriyi nasıl güncelleyeceğini** söylersin. Üç kavram:

- **DndContext**: sürükleme alanının çerçevesi; sensörler ve onDragEnd burada.
- **SortableContext**: sıralanabilir öge id'leri + strateji (verticalListSortingStrategy).
- **useSortable(id)**: her satıra ref, transform (animasyon) ve listeners (tutamaca bağlanır) verir.

```
const handleDragEnd = (e) => {  
  setActiveId(null);  
  const { active, over } = e;  
  if (!over || active.id === over.id) return;  
  const oldIndex = model.fields.findIndex(f => f.id === active.id);  
  const newIndex = model.fields.findIndex(f => f.id === over.id);  
  reorderFields(oldIndex, newIndex); // veri guncelleme store'da  
};
```

Üç ince ayar (drag'i 'oturtmak')

- **Tutamak ayrımı**: listeners'ı tüm satıra değil sadece grip ikonuna bağladık; böylece input'lara tıklanabilir.
- **activationConstraint {distance:6}**: 6px hareket etmeden sürükleme başlamaz — tıklama/sürükleme karışmaz.
- **DragOverlay**: sürüklenen satırın imleci takip eden bir kopyası — net görsel geri bildirim.
- **Mounted kapısı**: DndContext yalnızca mount sonrası render edilir; SSR hydration uyumsuzluğu önlenir.

Erişilebilirlik bonusu: KeyboardSensor sayesinde tutamağa Tab'la gelip Space ile tutup ok tuşlarıyla taşınabilir.

10. Test Stratejisi (vittest)

Toplam **40 test**, 6 dosya. Felsefe: **saf mantığı test et**, UI'ı gözle doğrula.

parse/layout/optimize/validate/segments saf fonksiyonlar olduğu için hızlı ve güvenle test edilir.

Dosya	Neyi doğrular
layout.test.ts	offset, padding, tail padding, dizi, boş struct, alignUp (9 test)
parser.test.ts	isim/alan, dizi, yorum, alias'lar, hatalar (9 test)
segments.test.ts	padding blokları, tail, renk indeksi, toplam=sizeof (5 test)
optimizer.test.ts	sıralama, padding azalması, stabilite, mutasyon yok (5 test)
validate.test.ts	boş/geçersiz/yinelenen, tekrar yok (5 test)
useStructStore.test.ts	reorder/add/remove/update/setName (5 test)

Öne çıkan teknik: **değişmez (invariant) testi** — 'tüm segment boyutları toplamı = totalSize'. Tek satırda mantığın bütününe doğrular; spesifik sayılara bağlı kalmaz.

11. Kapsam Sınırları ve Kararlar

Bilinçli kapsam kararları (juriye 'neden yapmadınız' sorusuna hazır cevap):

- **Standart-layout, paketlenmemiş struct** varsayıyoruz. #pragma pack, bit-field'lar, iç içe struct'lar kapsam dışı.
- **long platforma bağlı:** LP64 (Linux/macOS) 8 byte, Windows LLP64 4 byte. Biz 64-bit Unix modelini seçtik.
- **Pointer/referans yok:** sabit genişlikli değer tipleri üzerine odaklandık (eğitim amaçlı).

12. Sunum Q&A — Olası Sorular

S: Padding neden var?

C: CPU belleği hizalı bloklar halinde okur; hizasız erişim ya yavaştır ya da bazı işlemcilerde geçersizdir. Bu yüzden derleyici alanları hizalı offset'lere koyar ve araya boşluk (padding) ekler.

S: Hizalama (alignment) tam olarak nedir?

C: Bir tipin başlayabileceği offset kısıtı: offset, tipin hizalamasının katı olmalı (offset % align == 0). double'ın hizalaması 8'dir, yani 0/8/16... offset'lerine konabilir.

S: Tail padding neden gerekli?

C: Struct dizilerinde her elemanın hizalı kalması için. sizeof, struct'ın hizalamasının (en büyük alan hizalaması) katı olmalı ki arr[i+1] doğru hizalansın.

S: Struct'ın hizalaması nasıl belirlenir?

C: Üyelerin en büyük hizalamasıdır. Örn. bir double içeriyorsa struct'ın hizalaması 8 olur.

S: optimizeStruct nasıl çalışır, her zaman optimal mi?

C: Alanları hizalamaya göre büyükten küçüğe dizer. Bu basit greedy, sabit genişlikli tipler için pratikte padding'i en aza indirir; ara padding'i sıfırlar, geriye yalnızca kaçınılmaz tail padding kalabilir.

S: long kaç byte?

C: Platforma bağlı: 64-bit Linux/macOS'ta 8, 64-bit Windows'ta 4. Biz Unix (LP64) modelini varsaydık ve long'u int64_t'ye eşledik.

S: Neden her alanın bir id'si var, index yetmez mi?

C: Index sıra değişince kayar, isim değişebilir. Stabil id; drag-drop (SortableContext) ve diff için şarttır.

S: Hydration hatası neydi, nasıl çözdünüz?

C: Date.now() tabanlı id'ler sunucu/tarayıcıda farklı üretiliyordu. Başlangıç alanlarına sabit id verdik ve dnd-kit'i yalnızca mount sonrası render ettik.

S: Neden Zustand, neden Context değil?

C: Zustand hafif, az boilerplate ister ve seçici (selector) ile gereksiz render'ları önler. İki slice'ın paylaştığı tek buluşma noktası olarak sade kalır.

S: Testler tam olarak neyi garanti ediyor?

C: Saf mantığın doğruluğunu: offset/padding hesapları, parser kuralları, optimize'ın padding'i azaltması, doğrulama kuralları ve store action'ları. Refactor yaparken kırmadan ilerlemeyi sağlar.

13. Demo Senaryosu (adım adım)

Sunumda řu akıřı göstererek tüm slice'ı 60 saniyede anlatabilirsin:

- 1 Hazır Player struct'ı ile aç; bellek haritasını ve sizeof=16, padding=3'ü göster.
- 2 health'in tipini deęiřtir / yeni alan ekle — haritanın anında yeniden çizildięini göster.
- 3 Bir alanı tutamacından sürükle — sıra ve padding'in deęiřtięini vurgula (DragOverlay ile).
- 4 Kötü sıralı bir struct yapıştır (bool, double, bool) — sizeof=24/padding=14.
- 5 'Optimize et'e bas — sizeof 16'ya, padding 6'ya düşer; 'neden'ini anlat.
- 6 Zoom slider'ını çek — orantılı büyütme + yatay kaydırma.
- 7 Yinelenen alan adı ver — doğrulama uyarısını göster.

Özet: uçtan uca akıř — yapıştır (alias'lı parse) → düzenle/sürükle → optimize et → canlı, zoom'lanabilir bellek haritası — doğrulama ve testlerle desteklenmiř.

Bu belge Person A slice'ı içindir. Merge sonrası tüm pipeline'ı (Person B'nin versiyonlama/diff/export kısmı dahil) kapsayan bir final sürüm hazırlanacak.